FALL SEMESTER 2026–2027

BACSE202 – DATABASE MANAGEMENT SYSTEMS (DBMS)
DIGITAL ASSIGNMENT

AI POWERED FINANCIAL FRAUD DETECTION

Technical Project Report

Submitted by
VINODD V – 25BCE1221
VARUN KRISHNA – 25BCE1530
ARJUN KARTHIKEYAN – 25BCE1467

Faculty: Dr. S. Geetha
Slot: D1 + TD1
Class: CH202670101707
Date: 3.11.2026

ABSTRACT

Financial fraud is a major concern in digital banking because unusual transactions, compromised devices and suspicious beneficiary relationships can cause financial loss. This project presents an academic banking prototype named NeoBank that combines role-based banking operations with graph-based transaction monitoring. The application provides separate workspaces for Admin, Manager, Teller, Fraud Analyst, Support and Customer roles. Customers can manage beneficiaries, view account activity and initiate demo fund transfers. Staff roles receive controlled access to banking, customer service and investigation functions.

The backend is implemented using Python FastAPI and communicates with a Neo4j graph database. Banking entities such as Customer, Account, Transaction, Beneficiary, Login, Device, IPAddress, FraudCase and FraudFactor are represented as nodes connected by relationships. A demonstrative rule-based risk-scoring workflow considers indicators such as unfamiliar device/IP activity, unusually large transfers, recently added beneficiaries and transaction patterns. Depending on the configured score thresholds, a transfer can be marked SUCCESS, PENDING_OTP or BLOCKED. OTP and deposits are simulated for academic demonstration and are not connected to real banking rails.

The project demonstrates how graph relationships can support the inspection of connected banking activity and how role-based access can separate customer operations from administrative and fraud-investigation functions.

Keywords

Financial Fraud Detection, Neo4j, Graph Database, FastAPI, Cypher, Role-Based Access Control, Transaction Monitoring, Risk Scoring, OTP Verification.

TEAM DETAILS

Student Name	Register Number	Contribution (summary)
Vinodd V	25BCE1221	Application integration, database connectivity, transaction monitoring and report coordination.
Varun Krishna	25BCE1530	Application features, role workflows and testing.
Arjun Karthikeyan	25BCE1467	Database modelling, fraud-monitoring features and documentation.

1. INTRODUCTION

Digital banking enables customers to transfer money, manage beneficiaries and monitor accounts through online applications. These services also introduce security challenges: transactions may originate from unfamiliar devices, unusual IP addresses, newly registered beneficiaries or patterns that differ from a customer's normal activity. A banking system therefore needs both transaction processing and a way to inspect related entities and events.

NeoBank is an academic prototype designed to demonstrate banking operations and fraud-risk monitoring using a graph database. Unlike a purely tabular representation, a graph database makes relationships between customers, accounts, transactions, devices, IP addresses and fraud indicators directly navigable.

1.1 Problem Statement

A conventional banking workflow may store customer, account, transaction and login information in separate records. Identifying suspicious connections across these records can require multiple joins and additional analysis. The project addresses the need for a prototype that links banking entities and evaluates transactions using multiple configurable risk indicators while enforcing role-specific access.

1.2 Objectives

- Develop a web-based banking prototype with separate dashboards for six roles.
- Store banking entities and their relationships in Neo4j.
- Support customer account views, beneficiary registration and simulated fund transfers.
- Evaluate transfers using configurable risk indicators and risk-score thresholds.
- Demonstrate SUCCESS, PENDING_OTP and BLOCKED transaction paths.
- Provide staff workspaces for customer, employee, transaction, support and fraud-case management.
- Record login/device/IP context where available and make related graph records inspectable.

1.3 Scope

The scope covers a local academic prototype, role-based application navigation, graph persistence, simulated deposits and OTP, transaction risk evaluation and staff investigation screens. It does not include real payment-network integration, real SMS delivery, production identity verification or validated banking-grade fraud prediction.

2. EXISTING SYSTEMS

Existing digital banking platforms commonly provide online account access, beneficiary management, fund transfers, transaction history, authentication and security notifications. Commercial fraud-monitoring platforms may additionally correlate transactions, customer profiles, devices and network signals. The following public resources are included as background references; the project is an educational prototype and is not affiliated with these providers.

System / Resource	Relevant capability	URL
SBI Internet Banking	Online banking, transfers and account services	https://retail.onlinesbi.sbi/
HDFC Bank NetBanking	Account access, beneficiary and transfer workflows	https://www.hdfcbank.com/
Neo4j	Graph database and Cypher query language	https://neo4j.com/docs/
FastAPI	Python API framework used for backend services	https://fastapi.tiangolo.com/

The screenshots in this report document the developed prototype interface and its observed workflows; they are not screenshots of the external systems listed above.

3. REQUIREMENTS AND PROPOSED SYSTEM

3.1 Functional Requirements

- Authenticate users and route them to role-specific dashboards.
- Allow customers to view their own account details and recent activity.
- Allow customers to register beneficiaries and initiate demo transfers.
- Evaluate transfers and apply configured SUCCESS, PENDING_OTP or BLOCKED outcomes.
- Allow administrators to view customers, accounts, employees, transactions and fraud cases.
- Provide Manager and Teller banking workspaces with role-appropriate operations.
- Allow Fraud Analysts to inspect suspicious transactions, risk factors and case status.
- Allow Support staff to view and update support tickets.
- Persist application entities and relationships in Neo4j.

3.2 Non-Functional Requirements

- Usable browser-based interface with clear role navigation.
- Backend/database connectivity and consistent persistence.
- Role-based authorization for protected workspaces.
- Readable audit and investigation information.
- Prototype-level security warnings and clear disclosure of simulated functions.

3.3 Proposed Architecture

The browser-based frontend communicates with the FastAPI backend through HTTP API requests. The backend performs authentication, role checks, transaction processing and risk evaluation, and uses the Neo4j driver to execute Cypher queries. Neo4j stores nodes and relationships for banking, security and support information.

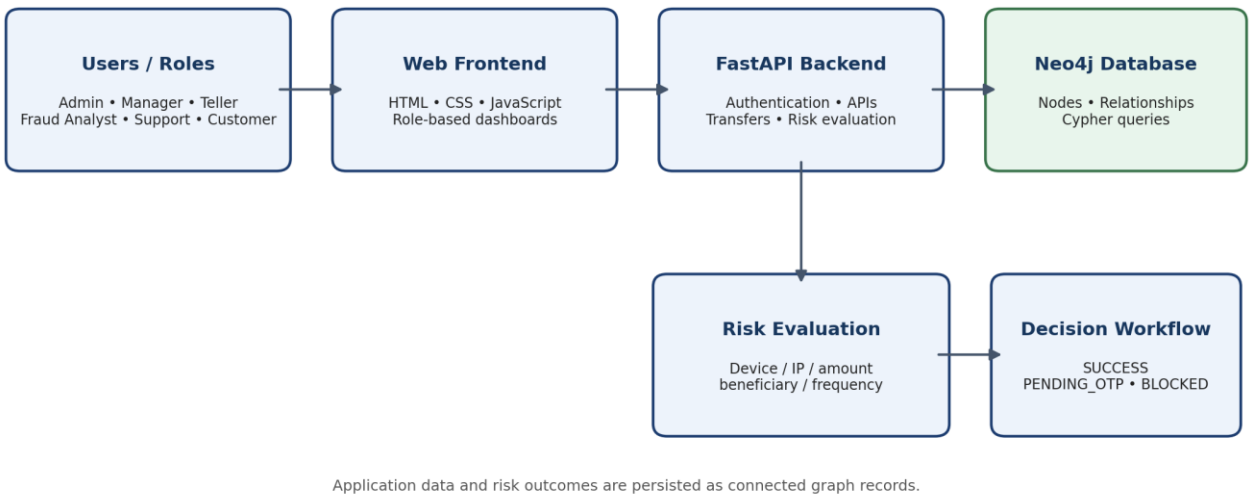


Figure 1. Proposed application architecture.

3.4 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and role-specific dashboards.

Backend	Python, FastAPI	Authentication, APIs, transaction and risk workflows.
Database	Neo4j	Graph persistence for banking entities and relationships.
Query language	Cypher	Create, connect, update and retrieve graph records.
Runtime / access	Uvicorn; browser over local network	Runs the API and serves application access.

4. DATA MODEL: ER-STYLE GRAPH MODEL AND SCHEMA

Neo4j is a property graph DBMS rather than a relational DBMS. Therefore, the model is presented as a graph-oriented conceptual ER representation: entities are represented by labelled nodes and associations are represented by directed relationships. The diagram below shows selected core entities and relationships; the complete database also contains the additional labels and relationship types listed in the schema inventory.

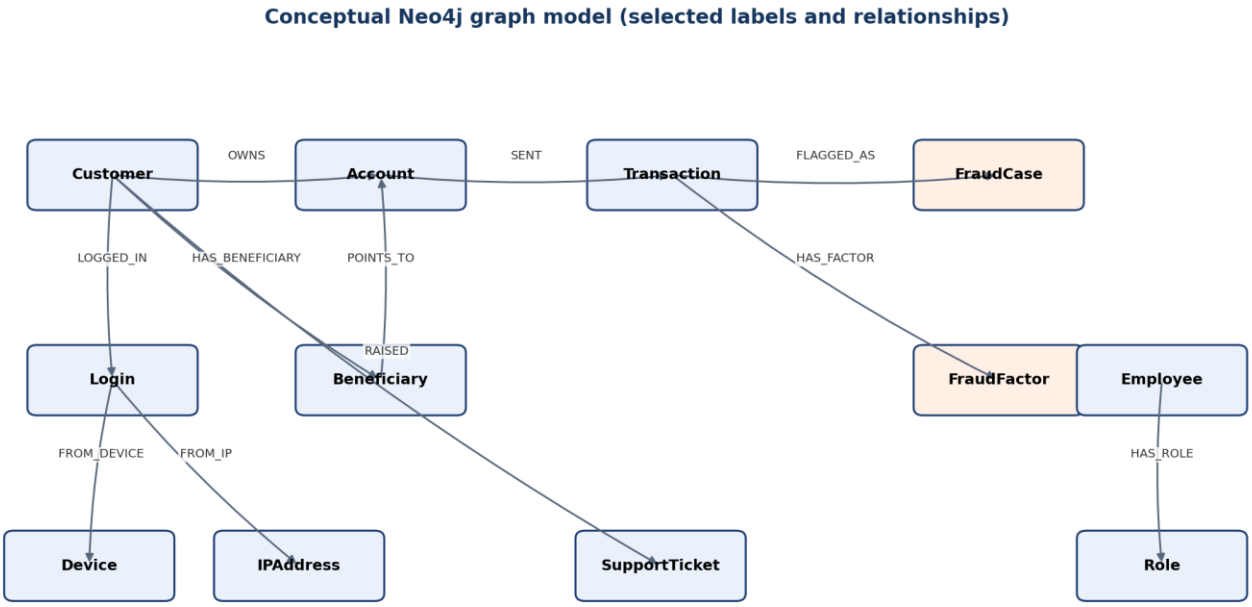


Figure 2. Conceptual graph model showing selected nodes and relationships.

4.1 Node Labels

Node label	Purpose
Account	Account number, balance, status and account attributes.
AuditLog	Records important application and administrative actions.
BankCard	Stores bank-card related information.
BankPolicy	Stores policy information used by the application.
Beneficiary	Stores beneficiary account, IFSC and nickname details.
Branch	Represents a bank branch and its IFSC/details.
Complaint	Represents customer complaint information.
Customer	Stores customer identity and contact information.
Department	Represents an employee department.
Device	Stores device fingerprint or device metadata.
Employee	Stores staff identity and employment details.
FraudCase	Represents a transaction investigation case and its status.
FraudFactor	Stores an individual risk indicator and its contribution.
IdCounter	Maintains counters used for sequential identifiers.
IPAddress	Stores observed IP address information.

Loan	Represents loan-related information.
Login	Stores login event details and timestamps.
Notification	Stores notifications associated with application activity.
OTPChallenge	Stores simulated OTP challenge information.
Role	Represents an assigned application role.
Session	Represents a user session.
SupportTicket	Stores customer support request details.
Transaction	Stores transaction amount, timestamp, status and risk score.

4.2 Relationship Types

Relationship	Meaning
ASSIGNED_TO	Links a staff member or case to an assigned owner, as used by the application.
AT_BRANCH	Connects an account or banking record to a branch.
FLAGGED_AS	Links a transaction to a fraud case.
FOR_LOGIN	Connects a session or OTP-related record to a login event.
FROM_DEVICE	Connects a login event to the device used.
FROM_IP	Connects a login event to an IP address.
HAS_BENEFICIARY	Connects a customer to a saved beneficiary.
HAS_FACTOR	Connects a transaction to its risk factors.
HAS_ROLE	Connects an employee to a role.
HAS_SESSION	Connects a user to a session.
IN_DEPARTMENT	Connects an employee to a department.
LOGGED_IN	Connects a customer to login events.
MADE_IN	Connects a transaction to associated login context, where recorded.
OWNS	Connects a customer to an account.
POINTS_TO	Connects a beneficiary to the destination account.
RAISED	Connects a customer to a support ticket.
RECEIVED_BY	Connects a transaction to the receiving account.
REGISTERED_BENEFICIARY	Records beneficiary registration by a customer.
SENT	Connects a sender account to a transaction.
USES	Connects a record or entity to a policy or resource as defined in the graph.

4.3 Selected Graph Patterns

- (Customer)-[:OWNS]->(Account): customer ownership of an account.
- (Account)-[:SENT]->(Transaction)-[:RECEIVED_BY]->(Account): transfer flow between accounts.
- (Customer)-[:HAS_BENEFICIARY]->(Beneficiary)-[:POINTS_TO]->(Account): saved beneficiary and destination account.
- (Transaction)-[:FLAGGED_AS]->(FraudCase): transaction associated with an investigation.
- (Transaction)-[:HAS_FACTOR]->(FraudFactor): risk factors contributing to transaction evaluation.
- (Customer)-[:LOGGED_IN]->(Login)-[:FROM_DEVICE]->(Device): login and device context.
- (Customer)-[:RAISED]->(SupportTicket): customer support request.

5. KEYS, CONSTRAINTS, FUNCTIONAL DEPENDENCIES AND NORMALIZATION

5.1 Keys and Uniqueness

Neo4j does not use primary-key and foreign-key declarations in exactly the same manner as a relational DBMS. The application uses identifier properties such as customer ID, account ID/account number, employee ID, transaction ID and ticket ID to identify nodes. Uniqueness constraints are the appropriate Neo4j mechanism for preventing duplicate identifier values. The exact constraints enabled in the running database should be verified using the Neo4j schema view or SHOW CONSTRAINTS.

Entity	Identifier / candidate key (conceptual)
Customer	customer.id
Account	account.id or account.acc_no
Employee	employee.id
Transaction	transaction.id
Beneficiary	beneficiary.id
FraudCase	fraudCase.id
SupportTicket	supportTicket.id
Login	login.id

5.2 Constraints

- Uniqueness: identifiers should uniquely identify Customer, Account, Employee, Transaction and other independently identified nodes.
- Existence / required attributes: application validation should require essential fields such as identifiers, account numbers and transaction amounts.
- Domain validation: amounts should be positive numeric values; role and status values should belong to the supported application options.
- Relationship integrity: backend Cypher operations should match the intended source and destination nodes before creating relationships.
- Authorization: role checks should prevent users from performing operations outside their assigned permissions.

5.3 Functional Dependencies

Functional dependencies are described here as a relational interpretation of the attributes represented by graph nodes. They help explain which attributes depend on an entity identifier, although Neo4j itself stores the model as a property graph.

- Customer_ID → Customer_Name, Email, Phone
- Account_ID → Account_Number, Balance, Status, Branch_ID
- Employee_ID → Employee_Name, Email, Phone, Role_ID
- Transaction_ID → Sender_Account, Receiver_Account, Amount, Timestamp, Status, Risk_Score
- Beneficiary_ID → Customer_ID, Destination_Account, IFSC, Nickname
- FraudCase_ID → Transaction_ID, Case_Status, Investigation_Note

5.4 Normalization Discussion

The project uses a graph data model, so relational normalization forms are not directly imposed on Neo4j nodes and relationships. Conceptually, the data is separated into distinct entities such as Customer, Account, Transaction, Beneficiary, Employee and FraudCase rather than repeating all details inside a single transaction record. This separation reduces unnecessary duplication and supports independent relationships. In relational terms, the conceptual entities can be mapped to tables with identifiers and foreign-key references; a formal 1NF/2NF/3NF decomposition would apply to that relational mapping, not to the native graph structure.

6. IMPLEMENTATION DETAILS

6.1 DBMS and Connectivity

Neo4j is used as the primary database. The FastAPI backend connects through the Neo4j Python driver using a Bolt connection URI, database username and password configured for the application environment. API handlers execute parameterized Cypher statements to create nodes, create relationships and retrieve application records. Credentials should be supplied through environment configuration rather than committed to a public repository.

6.2 Application Roles and Features

Role	Main capabilities evidenced by the prototype
ADMIN	System overview; customer and account directory; employee management; transactions; fraud cases; login events; network intelligence; audit trail; support tickets; Neo4j schema; open customer account.
MANAGER	Branch overview; customer directory; account oversight; employee directory; transactions; fraud review; login events; network intelligence; support tickets; open customer account.
TELLER	Branch desk; customer directory; account and transaction views; open customer account; demo deposit.
FRAUD_ANALYST	Fraud overview; transactions; fraud cases; login events; network intelligence; audit trail; Neo4j schema.
SUPPORT	Customer directory and support-ticket workflows, with access limited to support-related functions.
CUSTOMER	Own dashboard; fund transfer; beneficiary management; account statement; demo deposit; login/device security; device/IP graph; support tickets.

6.3 Fraud Risk Evaluation Workflow

The demonstrated transfer workflow computes a risk score from configured indicators. The project screenshots show a medium-risk transfer at 39.4% that requires simulated OTP verification and a high-risk transfer at 98.7% that is blocked. The risk score is demonstrative and rule-based in the current prototype; it should not be described as a clinically or banking-validated AI/ML prediction.

Risk score range	Prototype decision	Meaning
Below 35	SUCCESS	Transfer proceeds through the low-risk path.
35 to below 70	PENDING_OTP	Additional simulated OTP verification is required.
70 and above	BLOCKED	Transfer is blocked by the configured high-risk threshold.

The visible risk indicators include large share of account balance, recently added beneficiary and shared-device graph signals. The broader configured factors include new or unfamiliar device/IP activity, unusually high amounts, unusual timing and rapid repeated transfers. The exact score is produced by the backend's configured scoring logic.

6.4 Important Cypher Operations

Retrieve customer-owned accounts

```
MATCH (c:Customer)-[:OWNS]->(a:Account) RETURN c.id AS Customer_ID, c.name AS Customer_Name, a.acc_no AS
Account_Number, a.balance AS Balance;
```

Retrieve transfers between accounts

```
MATCH (a:Account)-[:SENT]->(t:Transaction)-[:RECEIVED_BY]->(b:Account) RETURN a.acc_no AS Sender, b.acc_no AS
Receiver, t.amount AS Amount, t.status AS Status;
```

Retrieve transactions and risk factors

```
MATCH (t:Transaction)-[:HAS_FACTOR]->(f:FraudFactor) RETURN t.id AS Transaction_ID, t.fraud_score AS
Risk_Score, f.name AS Fraud_Factor, f.points AS Risk_Points;
```

Retrieve employee roles

```
MATCH (e:Employee)-[:HAS_ROLE]->(r:Role) RETURN e.id AS Employee_ID, e.name AS Employee_Name, r.name AS
Assigned_Role;
```

Retrieve customer support tickets

```
MATCH (c:Customer)-[:RAISED]->(t:SupportTicket) RETURN c.name AS Customer, t.id AS Ticket_ID, t.subject AS
Subject, t.status AS Status;
```

7. IMPLEMENTED FEATURES

- Role-based login and role-specific dashboard navigation.
- Customer account creation and account overview.
- Beneficiary registration and beneficiary listing.

- Demo deposit workflow for recording simulated cash deposits.
- Fund transfer workflow with transaction status and risk score display.
- Simulated OTP verification for medium-risk transfers.
- Blocking of high-risk transfers based on the configured threshold.
- Admin and Manager transaction monitoring and fraud review.
- Fraud Analyst investigation workspace with case status and notes.
- Support ticket workflow linking customer and SupportTicket nodes.
- Login/device/IP context and Neo4j schema inspection.

8. TESTING, RESULTS AND DISCUSSION

8.1 Test Cases

ID	Test scenario	Expected result	Observed result
TC-01	FastAPI to Neo4j connection	Backend connects successfully.	PASS – FastAPI backend successfully connected to the Neo4j database.
TC-02	Valid customer login	Customer dashboard appears.	PASS – Customer successfully logged in and accessed the dashboard.
TC-03	Staff login for each role	Role-specific dashboard and permissions appear.	PASS – Staff roles successfully accessed their respective dashboards with role-based permissions.
TC-04	View owned account	Correct account information is displayed.	PASS – Customer account information and balance were successfully retrieved and displayed.
TC-05	Register beneficiary	Beneficiary node/relationship is stored.	PASS – Beneficiary was registered and the corresponding graph relationship was created.
TC-06	Low-risk demo transfer	SUCCESS path if score <35.	PASS – Low-risk transaction was processed with SUCCESS status.
TC-07	Medium-risk demo transfer	Simulated OTP path if score 35–69.	PASS – Medium-risk transaction triggered simulated OTP verification.
TC-08	High-risk demo transfer	BLOCKED path if score >=70.	PASS – High-risk transaction was blocked based on the configured risk threshold.
TC-09	Inspect transaction risk factors	Transaction and FraudFactor connections are visible.	PASS – Transaction risk factors and their relationships were retrieved from Neo4j.
TC-10	Raise support ticket	Customer is linked to SupportTicket by RAISED.	PASS – Support ticket was created and linked to the customer through RAISED.
TC-11	Run Cypher queries	Expected graph records appear in Neo4j Browser.	PASS – Selected Cypher queries executed and returned the expected graph records.

8.2 Results

The recorded test outcomes indicate that the backend-to-Neo4j connection, customer login, staff role dashboards, account retrieval, beneficiary registration, transfer risk paths, fraud-factor inspection, support-ticket creation and selected Cypher queries operated as expected in the tested prototype environment.

The screenshots show three transaction outcomes: a successful low-risk transfer at 16.1%, a medium-risk transfer at 39.4% with PENDING_OTP, and a high-risk transfer at 98.7% with BLOCKED status. The Fraud & Exception Workbench displays the corresponding transaction risk and case-management controls. These observations demonstrate the configured prototype workflow; they do not establish real-world fraud-detection accuracy.

8.3 Discussion

The graph model provides a direct way to inspect how customers, accounts, transactions, beneficiaries, login events, devices, IP addresses and fraud factors are connected. This is useful for investigating patterns that span multiple entities. Role-based dashboards separate administrative management, customer transactions, teller operations, support and fraud investigation. The prototype also demonstrates step-up verification for medium-risk activity and blocking for high-risk activity.

The main limitations are that deposits and OTPs are simulated, the scoring indicators are rule-based and demonstrative, and the application is not connected to real banking rails. The screenshots include a localhost loopback IP in one session and a LAN IP in another; device and IP values depend on the environment from which the user accesses the application.

8.4 Evidence Screenshots

The following figures are screenshots captured from the submitted prototype. They are placed with the corresponding feature descriptions. Figure numbering is continuous across the report.

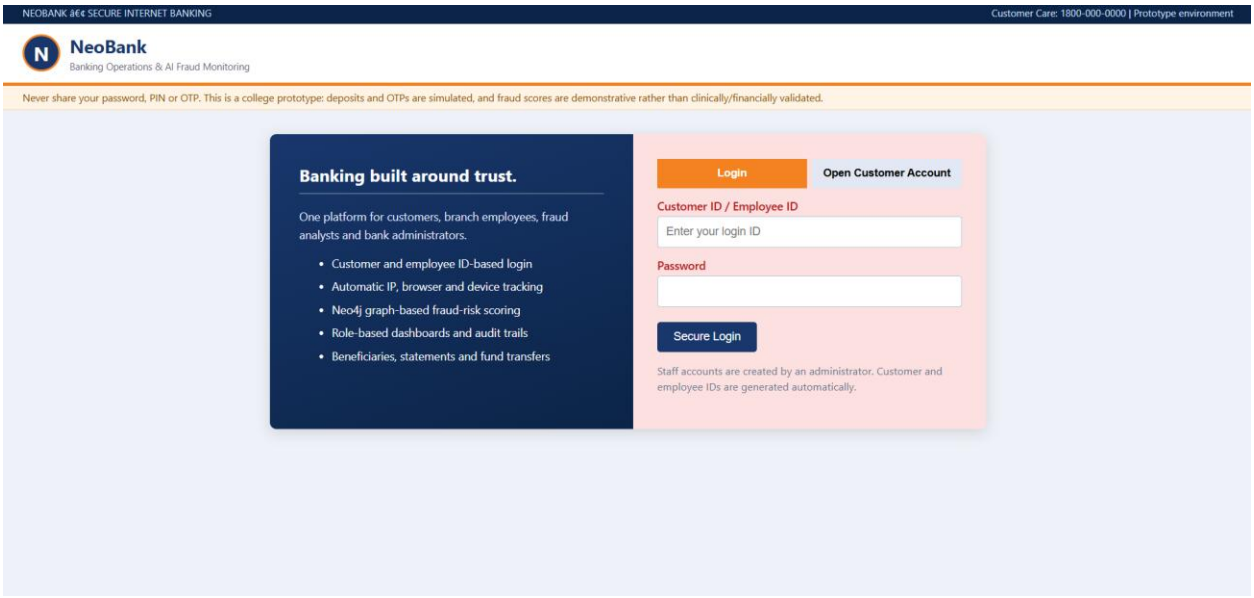


Figure 3. Login interface for customer and employee access.

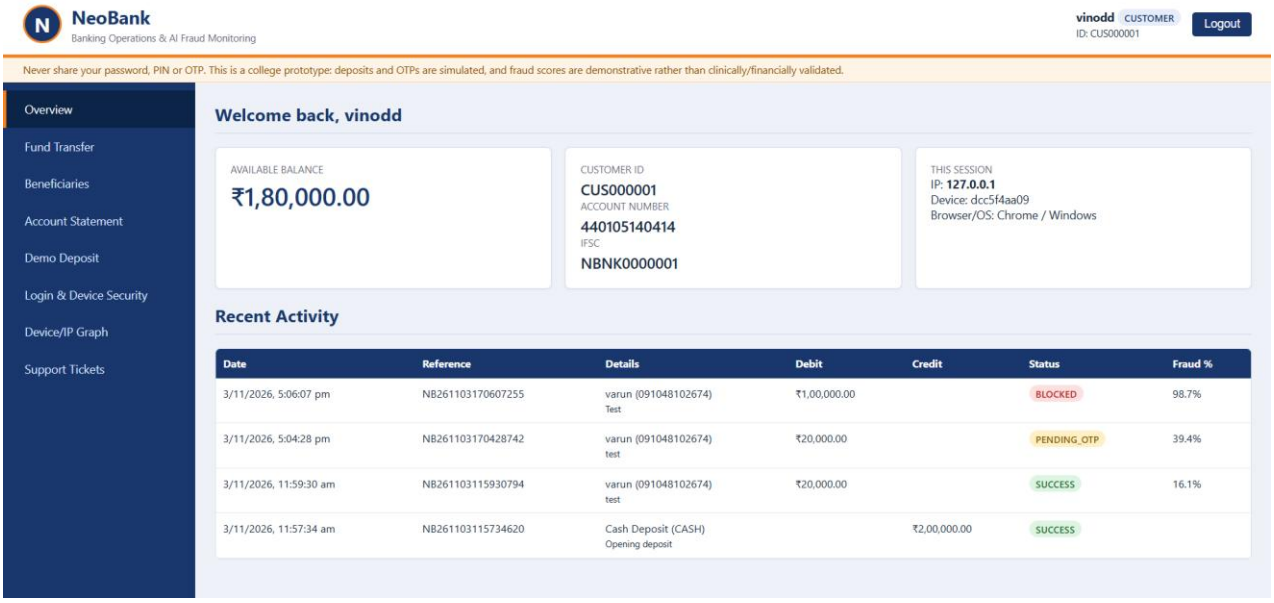


Figure 4. Customer dashboard showing balance, account details, session information and recent activity.

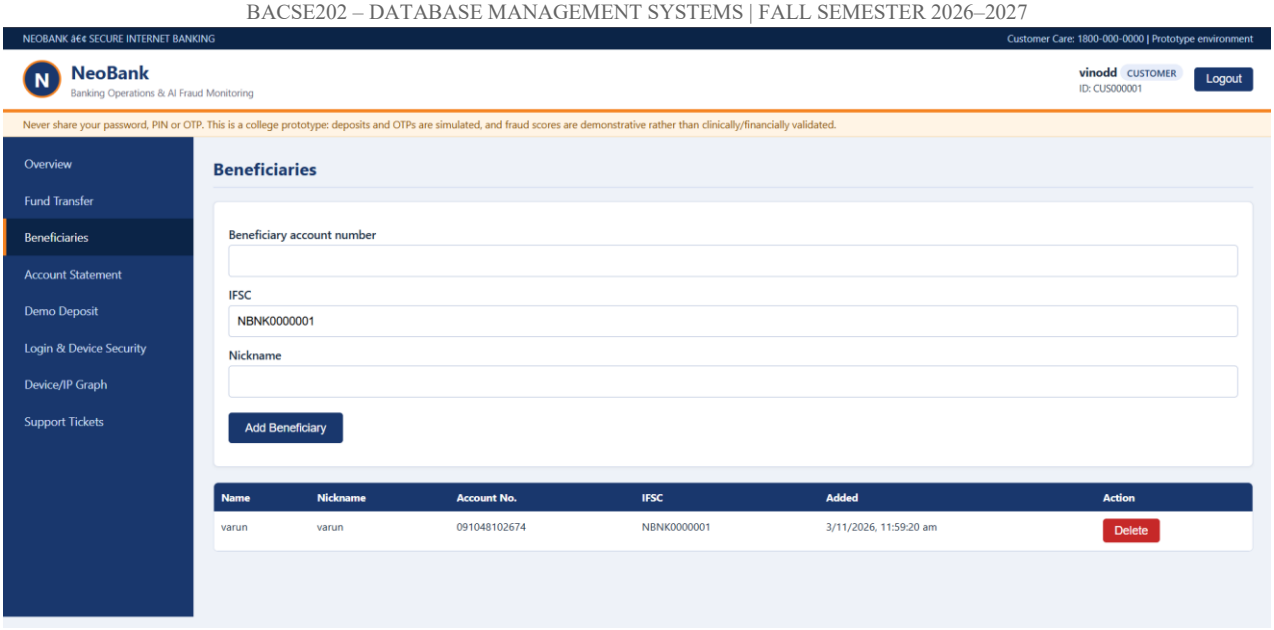


Figure 5. Beneficiary registration form and saved beneficiary list.

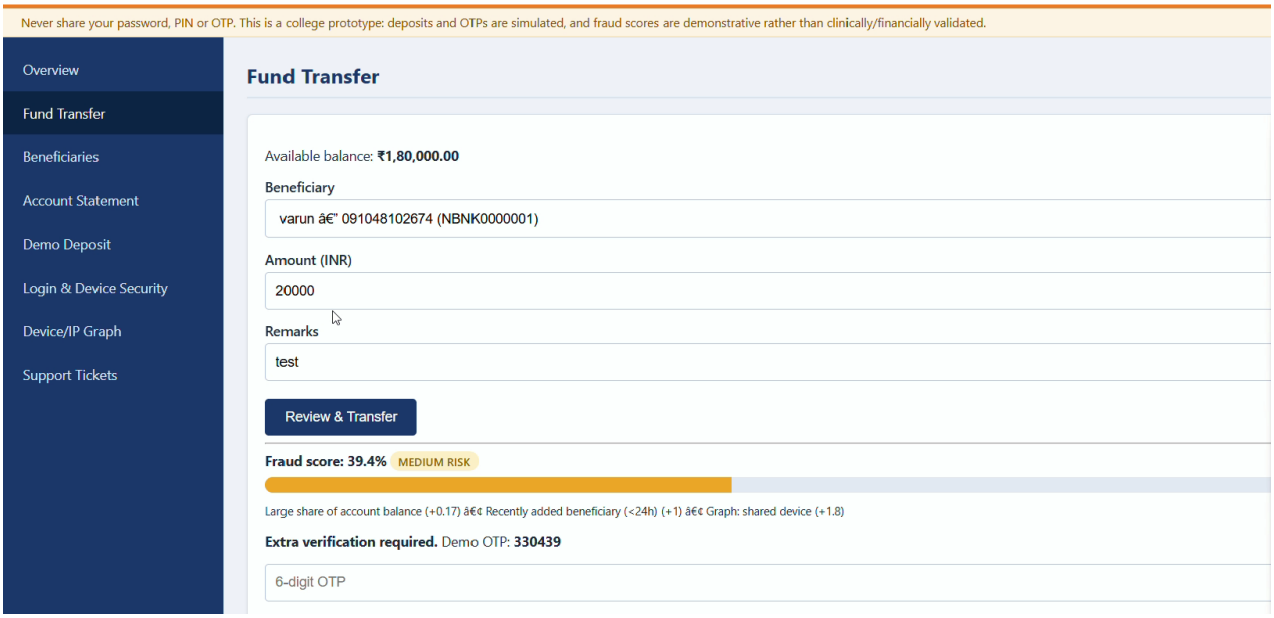


Figure 6. Medium-risk transfer showing 39.4% risk and simulated OTP verification.

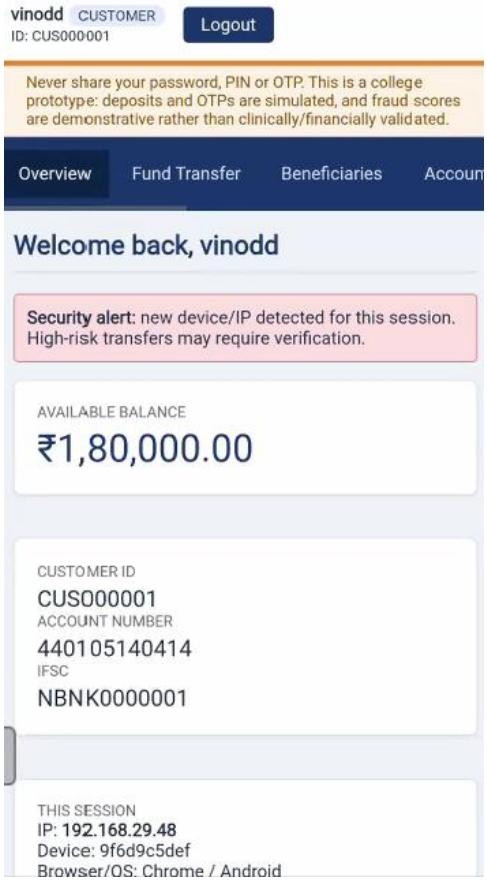


Figure 7. Customer dashboard accessed from a mobile device, demonstrating device/IP context.

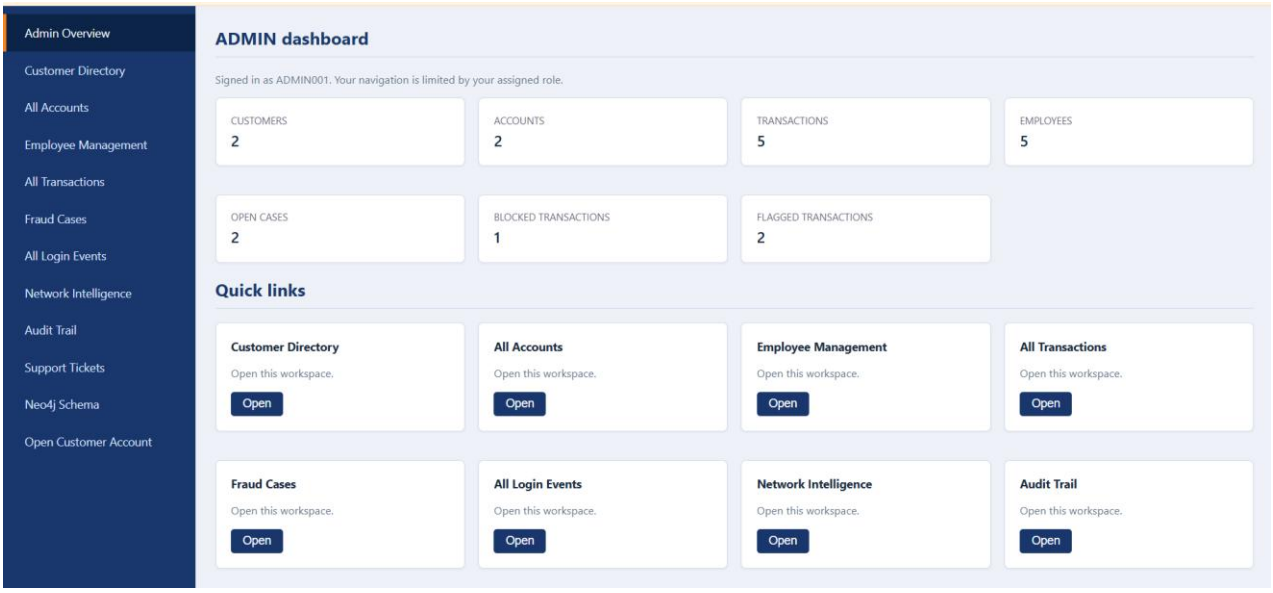


Figure 8. Admin dashboard showing customer, account, transaction, employee and fraud summary counts.

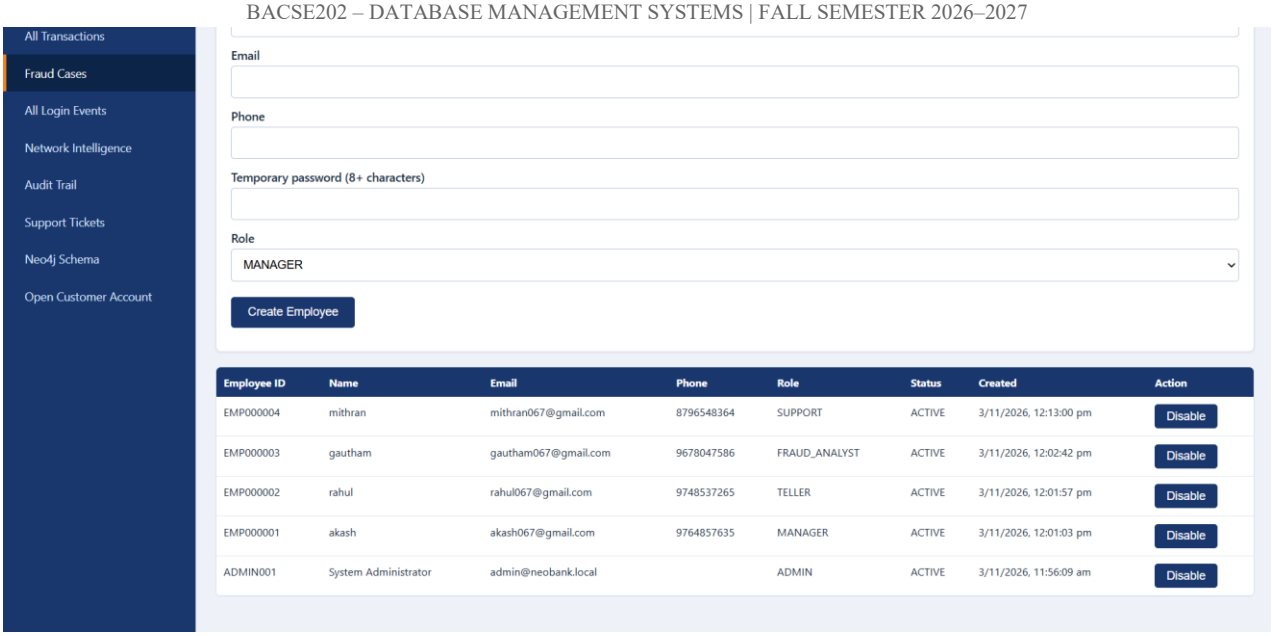


Figure 9. Employee management screen with staff roles and account status.

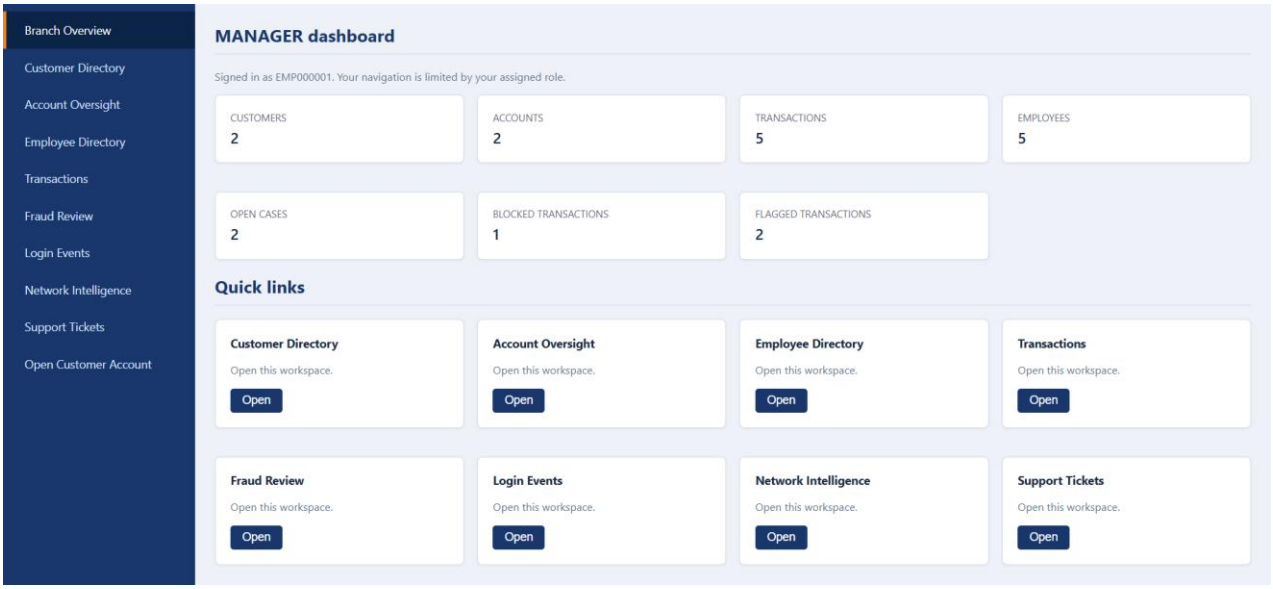


Figure 10. Manager dashboard with branch overview and role-specific workspaces.

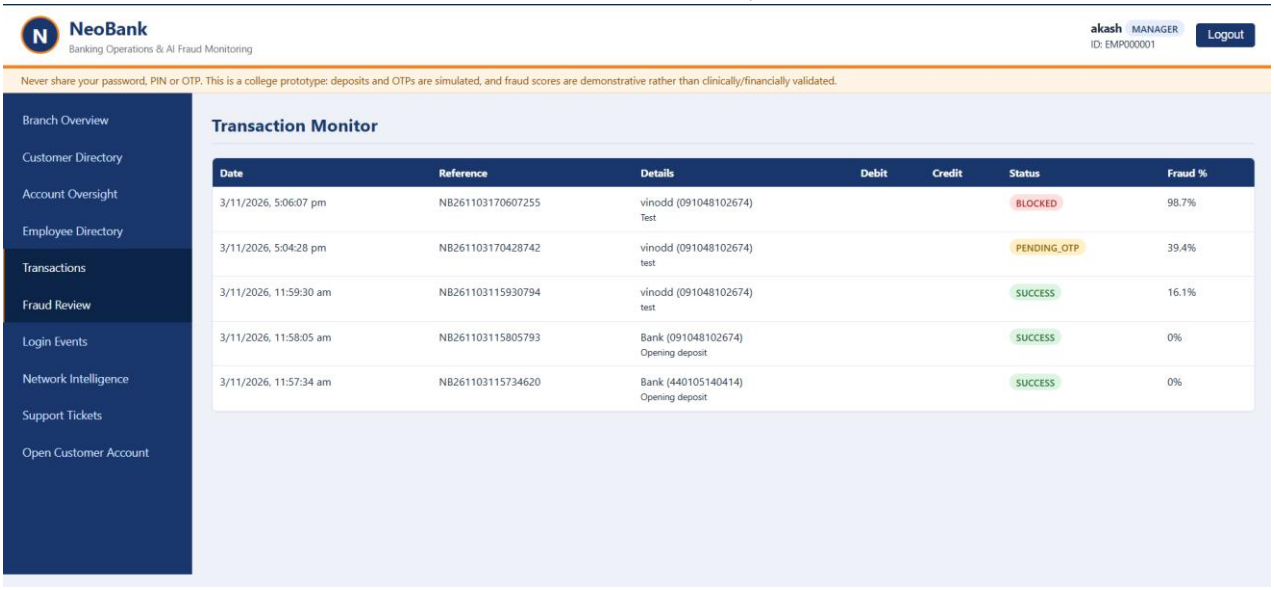


Figure 11. Manager transaction monitor showing SUCCESS, PENDING_OTP and BLOCKED records.

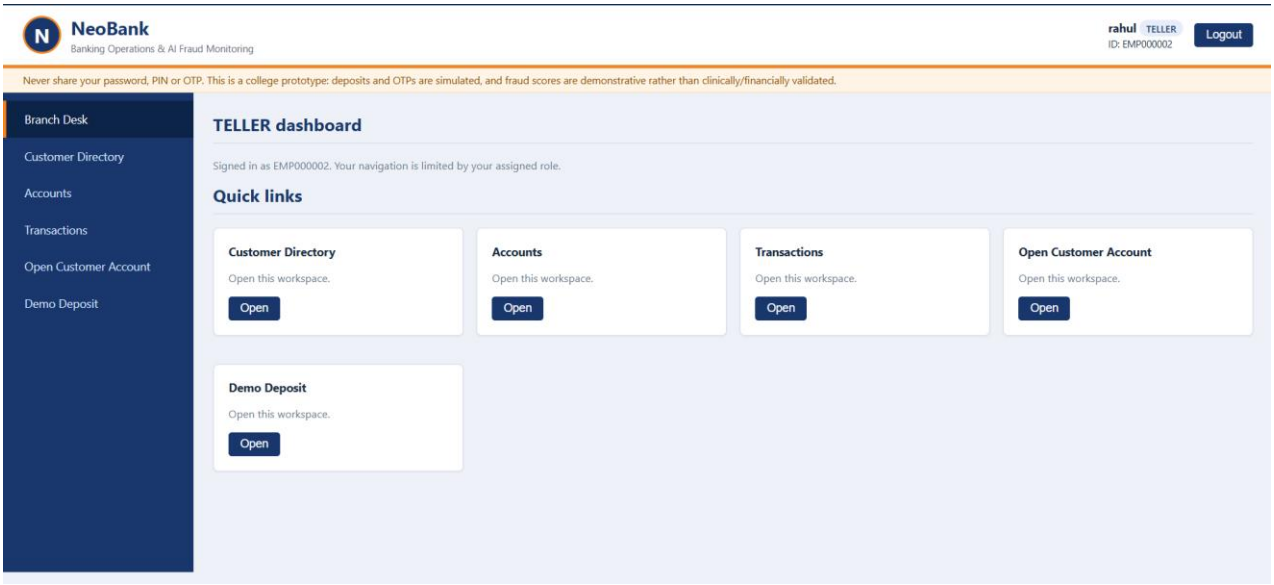


Figure 12. Teller dashboard showing branch-desk operations and demo deposit access.

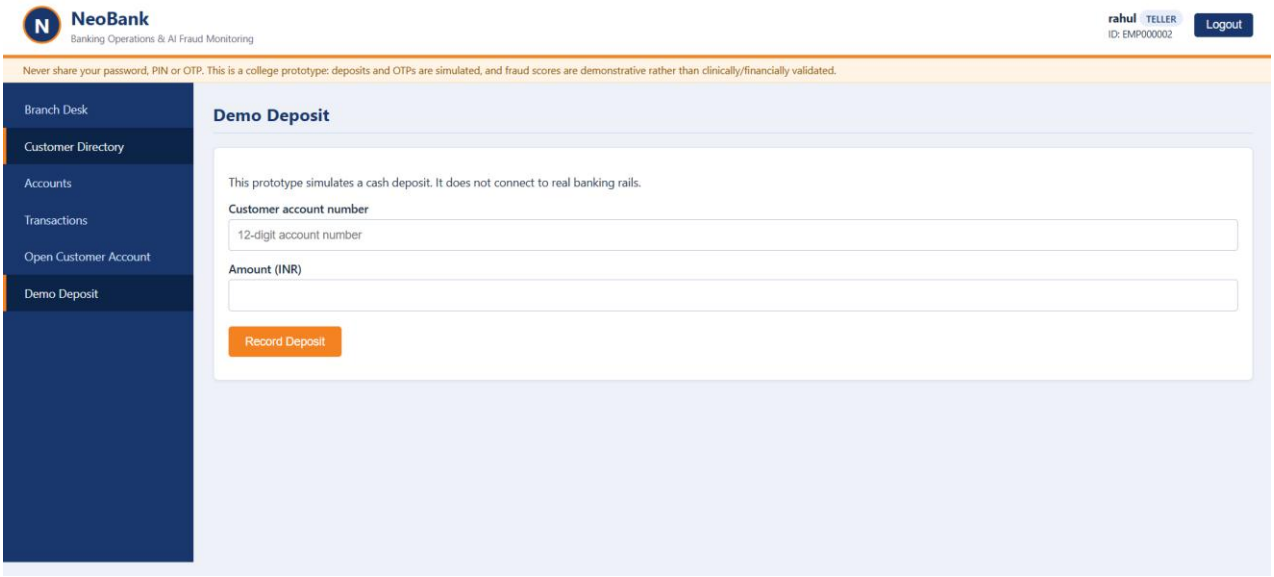


Figure 13. Demo deposit form; the interface explicitly identifies deposits as simulated.

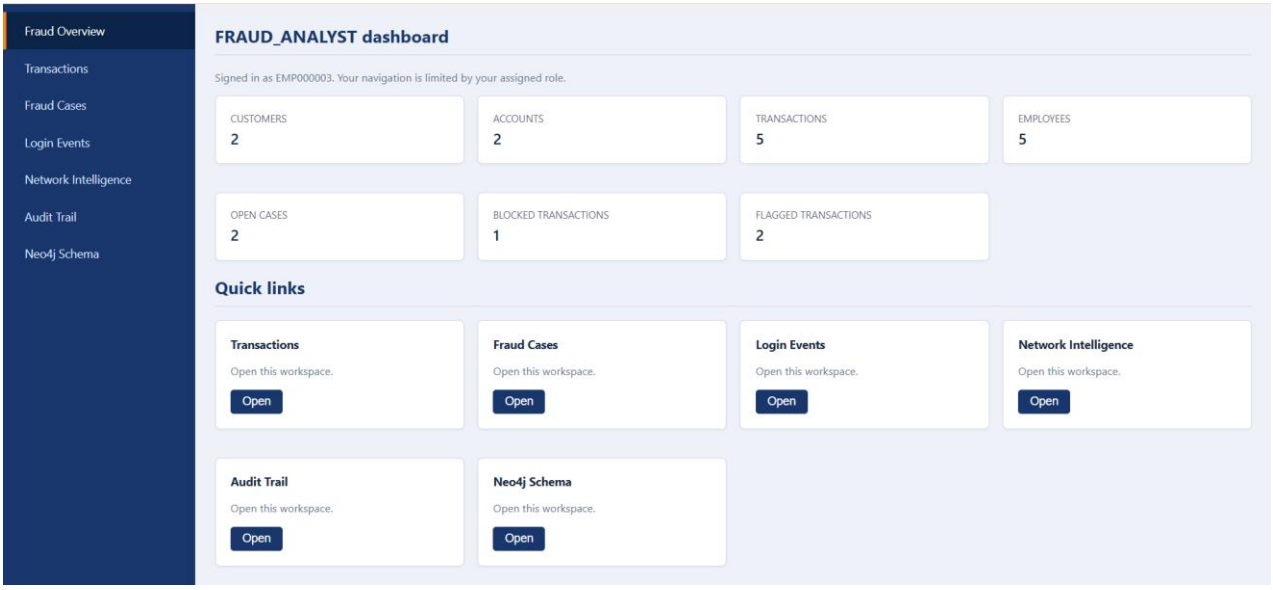


Figure 14. Fraud Analyst dashboard with monitoring summaries and investigation workspaces.

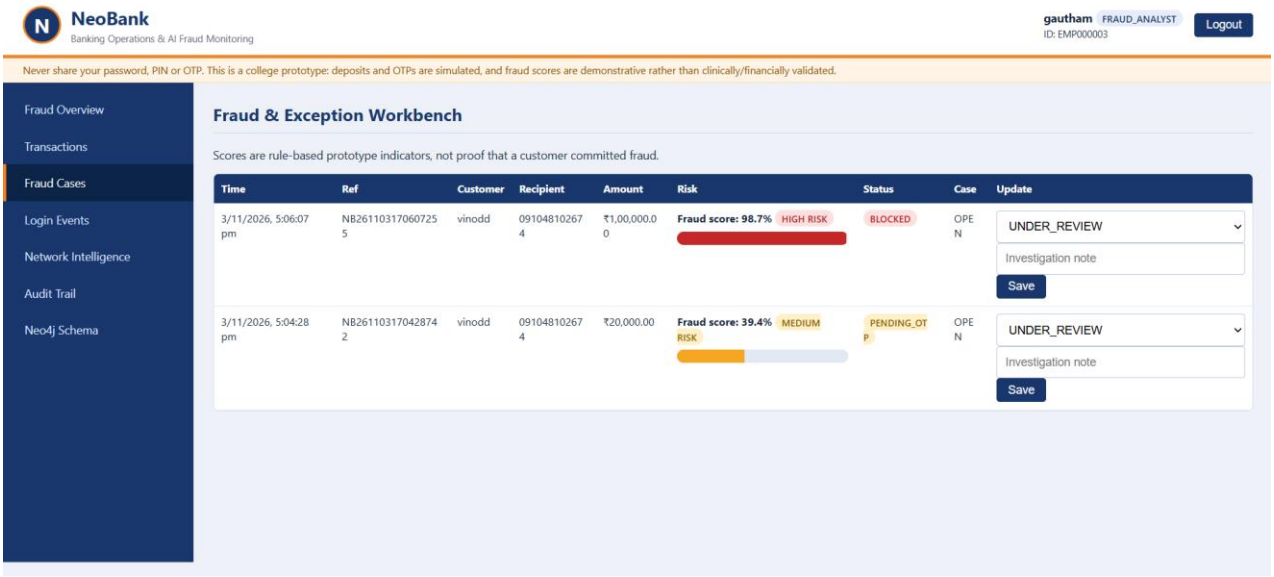


Figure 15. Fraud Analyst case workbench showing high- and medium-risk transactions, scores and case update controls.

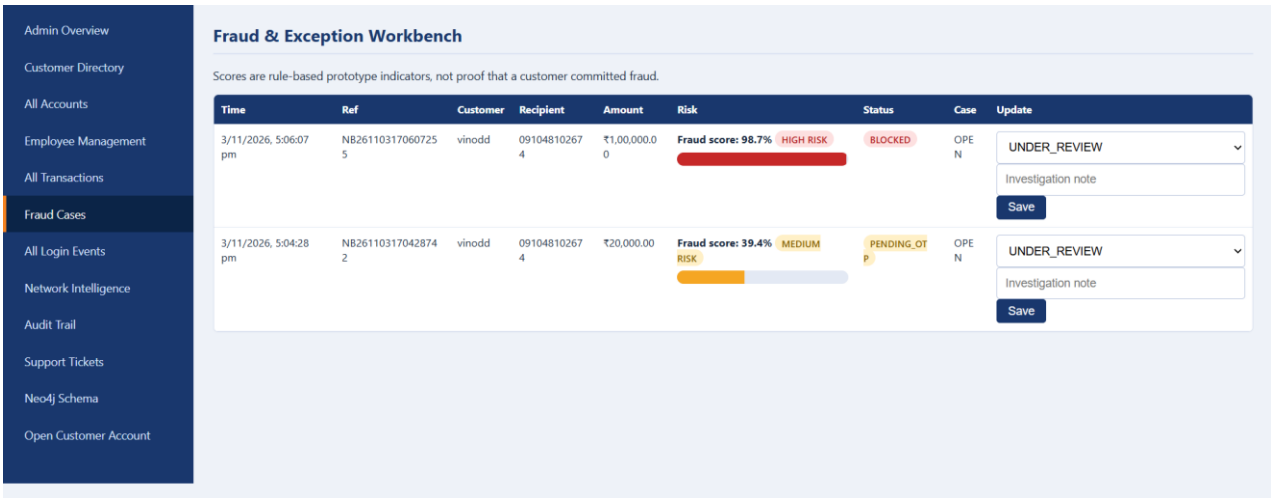


Figure 16. Admin fraud and exception workbench showing transaction risk scores and investigation status.

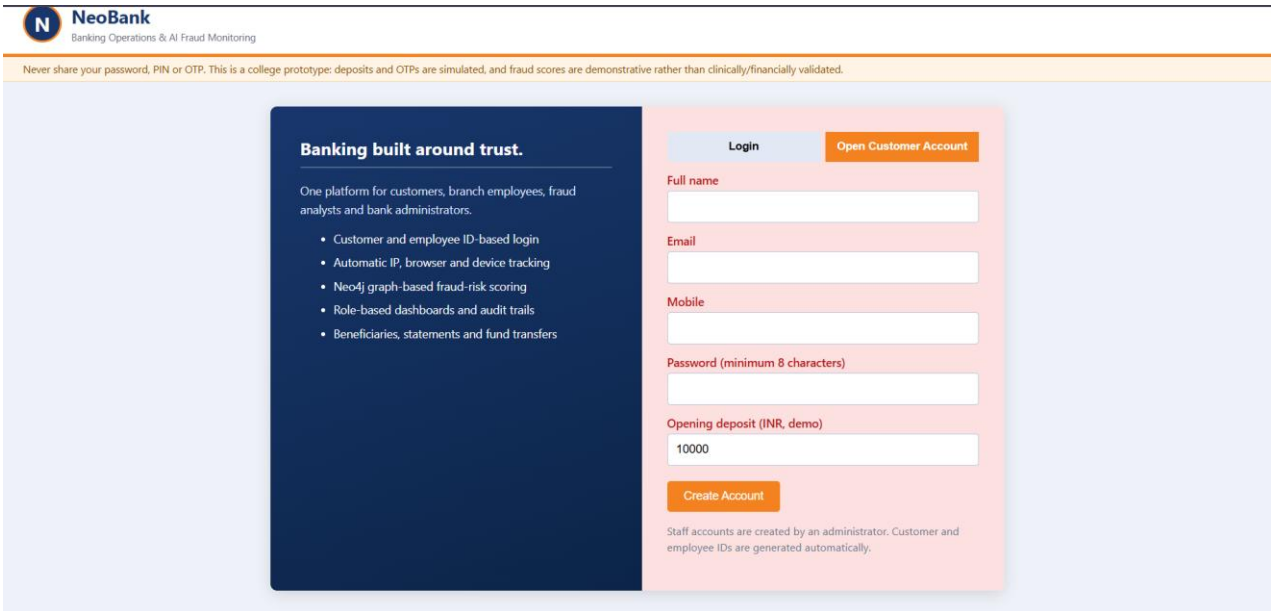


Figure 17. Open customer account form.

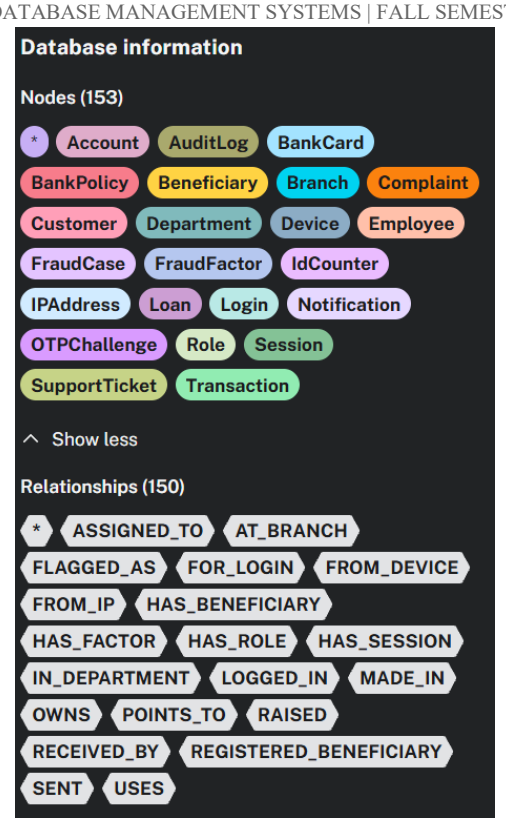


Figure 18. Neo4j database information panel showing node labels and relationship types.

9. CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Conclusion

The AI Powered Financial Fraud Detection project demonstrates a role-based banking prototype backed by a Neo4j graph database. It combines customer banking operations, staff dashboards, beneficiary management, demo transfers, transaction risk scoring and fraud-case investigation. The graph representation links key banking and security entities, allowing connected records to be inspected through Cypher queries. The observed test cases and screenshots demonstrate the intended application workflows in the academic prototype environment.

9.2 Limitations

- The project is an academic prototype and is not connected to real banking payment rails.
- Deposits and OTP verification are simulated; no real SMS OTP is sent.
- The current fraud score is based on configured rule-based indicators and is not validated as a production AI/ML model.
- Risk thresholds and factor weights require calibration using representative, labelled transaction data.
- Production deployment would require stronger security controls, privacy safeguards, audit review and regulatory compliance.

9.3 Future Enhancements

- Train and validate a machine-learning fraud classifier using an appropriate labelled transaction dataset.
- Evaluate precision, recall, F1-score, ROC-AUC and false-positive rates on held-out data.
- Integrate secure real OTP delivery and stronger multi-factor authentication.

- Add streaming transaction monitoring and alert notifications.
- Improve device fingerprinting and network intelligence with privacy-aware controls.
- Add production-grade encryption, secrets management, rate limiting, monitoring and backup/recovery.

10. REFERENCES

[1] Neo4j, “Neo4j Operations Manual and Cypher Manual,” Neo4j Documentation. [Online]. Available: https://neo4j.com/docs/. [Accessed: Nov. 3, 2026].

[2] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Nov. 3, 2026].

[3] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. [Accessed: Nov. 3, 2026].

[4] State Bank of India, “Personal Banking – Internet Banking.” [Online]. Available: https://retail.onlinesbi.sbi/. [Accessed: Nov. 3, 2026].

[5] HDFC Bank, “NetBanking.” [Online]. Available: https://www.hdfcbank.com/. [Accessed: Nov. 3, 2026].